

Session 5 Date: April 29, 2026 (2 hours)

Objective: Run performance experiments on the LiveJournal dataset and analyze speedup across thread counts.

Attempts made:

Added timing using chrono around each run and looped over 1, 2, 4, 8 threads so I could get everything in one run without recompiling.

Solutions/Workarounds:

```
auto start = chrono::high_resolution_clock::now();
BspSerial(g, bfs_serial);
auto end = chrono::high_resolution_clock::now();
cout << chrono::duration<double>(end - start).count() << " sec" << endl;
```

Commits: added timing and speedup measurements to main.cpp

Experiments/Analysis:

Algorithm	Serial	1 thread	2 threads	4 threads	8 threads
BFS	4.75s	4.76s (1.00x)	4.29s (1.11x)	3.97s (1.20x)	3.91s (1.21x)
SSSP	16.74s	14.98s (1.12x)	12.66s (1.32x)	11.71s (1.43x)	10.91s (1.53x)
CC	4.06s	4.19s (0.97x)	3.21s (1.27x)	3.30s (1.23x)	2.87s (1.41x)

SSSP got the best speedup at 1.53x with 8 threads which makes sense since it does the most work per round. BFS barely moved past 4 threads since the graph diameter is small and there just aren't enough rounds to make parallelism worth it. CC had a weird drop at 4 threads where it ran slower than 2, I think at that point the overhead of spawning more threads was just eating into whatever gains you'd expect. The 1 thread parallel times being slower than serial on BFS and CC was expected too since you are paying the full cost of thread management without getting any actual parallelism back.

Discussion Questions:

For the first question, `Process(tid, v, g)` only writes to the state of vertex `v`, so `dist[v]` for BFS and SSSP and `comp[v]` for CC. The parallel runner cuts the vertex range into fixed contiguous chunks and hands each chunk to one thread. Since no two threads share a chunk and `Process` only touches the current vertex, there is no way for two threads to write to the same memory location in the same round.

For the second question, SSSP scaled the best out of the three. It has the heaviest workload per round since every vertex has to scan its in-neighbors and compute weighted distances, so there is actually enough computation to justify splitting it across threads. BFS was the worst, barely hitting 1.21x at 8 threads. The LiveJournal graph has a small diameter so BFS wraps up in very few rounds and thread overhead ends up dominating. CC fell somewhere in the middle but the dip at 4 threads was a bit unexpected and points to spawning costs not being worth it at that thread count.

For the third question, since $\text{dist}[v]$ is permanent once set you could maintain a frontier of only the newly discovered vertices from the previous round instead of scanning all 4.8 million vertices every time. The frontier shrinks quickly as BFS progresses so this would save a lot of unnecessary work in later rounds. This approach does not carry over to SSSP or CC. In SSSP a vertex's distance can keep getting improved across many rounds as shorter paths are found, so you can never safely mark it as done. CC has the same issue since labels keep propagating until the whole graph converges, meaning every vertex stays active until the very end.

Learning outcome:

The speedup numbers were lower than I expected. Spawning threads every round is more expensive than it looks on paper, and on a graph with a small diameter like LiveJournal there just are not enough rounds for parallelism to really pay off. Looking back at it, SSSP being the slowest algorithm overall but the best at scaling makes a lot of sense given how much heavier each round is compared to BFS and CC.